FINTRUST

Transaction Insights and Risk Monitoring

System design and engineering record

Version 0.2 | 30 September 2026

Requirements, SDLC, user stories, database decisions, API contract, failure handling, service selection, testing and delivery.

Evidence boundary

The local foundation uses real PostgreSQL and simulated AWS services. Cloud deployment, automated risk scoring, notifications and scheduled reports are not complete. Architecture diagrams distinguish implemented foundations from the proposed target.

1. Executive overview and scope

FinTrust is a transaction-data quality and monitoring application. It accepts structured transaction exports, validates rows, stores accepted records in PostgreSQL, isolates rejected rows, and exposes account-level information through a Flask REST API and analyst dashboard. The application does not transfer money, authorize payments or make real compliance decisions.

The implemented release is a local foundation. PostgreSQL is real; the local practice command simulates S3 and DynamoDB through Moto. The daily risk insight is a labelled fixture. Automated scoring, compliance email delivery, scheduled reports and the final FIFO pipeline are incomplete. Their handlers fail explicitly instead of reporting fictional success.

The dashboard presents transaction count, accounts with activity, cross-border count, daily transaction frequency, totals separated by currency, an account-filtered ledger, pagination, date filters, CSV intake and rejection reasons. It avoids presenting a risk score as a confirmed fraud finding. A score is a review signal requiring a defined policy and human judgement.

2. SDLC and requirements traceability

The repository supports an iterative software development life cycle. A folder layout alone does not establish SDLC compliance: requirements, design decisions, tests, release evidence and operational reviews provide the traceability. Planning and design exist, implementation and local verification are active, and cloud deployment/operations acceptance remain open.

Phase	Project evidence	Exit condition
Initiation	Business problem, stakeholders, scope and cost constraint	Agreed capstone scope and assessable outcomes
Analysis	User stories, CSV contract, source-document conflicts	Accepted requirements and risk rules
Design	AWS architecture, ERD, activity/use-case/sequence diagrams, API contract	Reviewed boundaries and data ownership
Implementation	fintrust, frontend, SQL and infrastructure source	Reviewable incremental commits
Verification	Pytest, Moto, PostgreSQL checks and test-result artifacts	Passing relevant tests and recorded limitations
Delivery	CI and manually triggered API delivery workflow	Configured account, approvals and deployed smoke tests
Operation	Logs, quarantine, replay and alarm designs	Recovery, cost and security evidence
Improvement	Daily progress and upcoming features	Prioritised findings with verification evidence

User stories and acceptance criteria

ID	Story	Acceptance criteria and current evidence
US-01	As an operations analyst, I need CSV ingestion so exports can be checked consistently.	Valid schema loads; 1 MiB/5,000-row bounds enforced; custom upload test passes. Local-only upload adapter.

ID	Story	Acceptance criteria and current evidence
US-02	As a data steward, I need invalid rows isolated so valid records can proceed.	Rejected row retains source line and reason; mixed-validity tests verify valid/rejected counts.
US-03	As a reviewer, I need account/date filters so a transaction can be located.	UUID validation, parameterised SQL, stable date/ID ordering and pagination tests.
US-04	As a manager, I need currency-aware activity summaries so totals are meaningful.	Dashboard groups money by currency; SQL test prevents mixed-currency balances.
US-05	As an operator, I need safe replay so a duplicate event does not duplicate stored rows.	Source-version/row IDs for cloud ingestion; content/row IDs for local uploads; replay scope is documented.
US-06	As a maintainer, I need automated verification so broken changes are detected.	Unit/Moto tests, PostgreSQL integration checks, CI YAML and JUnit artifacts. Remote CI evidence pending.
US-07	As a compliance reviewer, I need explainable alerts for suspicious activity.	Planned: approved scoring rules, persisted decisions, SES integration and acknowledgement workflow.
US-08	As a project owner, I need controlled AWS costs and access.	No resource provisioning without account/cost review. IAM/KMS/network completion and billing evidence pending.

Functional requirements include ingestion, row isolation, relational integrity, read APIs, a review dashboard and traceable errors. Non-functional requirements include explicit input bounds, predictable error contracts, restricted browser origins, reproducible tests, private cloud database access and cost visibility. Numeric throughput, availability and cloud-latency objectives have not been validated.

3. Data sources, file contract and capacity

In the scenario, source files come from exports produced by a legacy banking ledger or transaction-processing platform. An operations analyst is the receiving actor. In this capstone, files are synthetic CSVs, including the supplied sample and compatible files produced for testing. No live bank feed, automatic NAS synchronization or external banking integration is connected.

Any filename can be used if the content meets the contract. The parser expects UTF-8, an optional UTF-8 BOM, comma-separated fields and a single header row. Quoted fields follow Python CSV parsing rules. Required headers are case-sensitive: account_id, amount, tx_date and tx_type. Optional fields are currency, merchant and country. Extra named columns are currently ignored; duplicate headers and extra unlabelled row values are rejected. Excel workbooks, JSON, semicolon-delimited exports and arbitrary CSV layouts require an adapter.

Field	Contract	Storage/result
account_id	UUID; referenced account already exists	Foreign key to accounts
amount	Finite positive decimal; at most two decimal places; below 10^13	NUMERIC(15,2)
tx_date	ISO date/timestamp; missing timezone interpreted as UTC	TIMESTAMPTZ
tx_type	debit, credit, transfer or fee	Allowlisted transaction type
currency	ZAR, USD, EUR, GBP or KES; defaults to ZAR	Currency is retained, not converted

Field	Contract	Storage/result
merchant	Optional text; database maximum 200 characters	Nullable merchant description
country	Optional; defaults to ZA	Any non-ZA value marks cross_border; country-code validation remains a limitation

Current engineering limits are 1 MiB of CSV bytes and 5,000 data rows per file. Files over either bound fail before database inserts. Flask also limits the entire multipart request to 1 MiB plus 64 KiB for form overhead. The UI allows selection of up to five files and submits them sequentially. Each POST contains one file. An error in one file does not undo a previous successful file in that browser batch.

The S3 handler iterates over event records. Across invocations, concurrency depends on deployed Lambda settings and database capacity. There is no measured maximum number of files per hour and no claim of unlimited capacity. The bounded file size is a guardrail, not a benchmark. Larger workloads require chunking, queue backpressure, concurrency controls and load tests.

Invalid rows and replay

Row validation errors and row-level database data/integrity errors are recorded in quarantine while acceptable rows continue. Quarantine records contain a source line, a reason and the submitted row. Cloud ingestion writes JSON Lines under a version-specific S3 quarantine key. Local browser uploads use simulated S3 and return the first 50 rejection records for review; the simulation disappears when the server stops. The CLI sample exercise also exports a local quarantine file.

A missing required header, duplicate header, invalid encoding or an exceeded file bound is a file-level failure. A local upload returns an error; the AWS handler raises the error for the configured asynchronous failure policy. A database/S3 outage is not treated as a bad transaction row. It raises a retriable invocation failure; after the configured retries, the ingest DLQ preserves the source event.

Source-object-version plus row-number IDs protect cloud-event replay. The local browser path uses a SHA-256 digest of file bytes plus row number, so the same bytes under a different filename produce the same local upload IDs. These are distinct source-identity schemes. Reuploading changed bytes, uploading through a different path or correcting an entire file can create new ledger IDs for previously accepted business transactions. A source-provided transaction ID, reconciliation process and explicit correction workflow are required before production.

4. Database and SQL design

FinTrust uses purpose-specific storage rather than treating every datastore as interchangeable. PostgreSQL is the relational system of record: foreign keys, constraints, joins and transactional writes preserve ledger relationships. DynamoDB serves a known lookup pattern, account_id plus score_date, without joining five tables. S3 stores immutable source-file versions and quarantine/report objects; it is not a substitute for transactional integrity.

Store	Appropriate responsibility	Trade-off
PostgreSQL / RDS	Customers, accounts, ledger, risk events and alerts; reporting joins	Connection limits, indexing, migrations and transaction boundaries require care
DynamoDB	Daily operational score lookup by account/date	No cross-store foreign keys; aggregation and reconciliation need explicit design
S3	Raw exports, quarantine and report files	Objects are not relational rows; retention/versioning differs from SQL deletes

The ERD contains customers, accounts, transactions, risk_events and alerts. Each account belongs to one customer, each transaction belongs to one account, each risk event references one transaction, and each alert references one risk event. One parent may have zero or many children. Separate entities reduce update anomalies for the stated dependencies; this

is not a full double-entry banking ledger.

The SQL package includes DDL, 200 synthetic seed transactions across five accounts, a rolling risk CTE, a window-function report, views and flag_account. The risk query counts transactions with qualifying risk events rather than counting repeated decisions as separate transactions. The High threshold is 70 in the current prototype; approval of business scoring rules remains outstanding.

The running-balance query is a zero-opening-balance net-movement exercise, partitioned by account and currency. Credits add; debit, transfer and fee subtract. Transfers are therefore assumed outgoing. Opening balance, incoming/outgoing transfer direction and foreign-exchange conversion are not modeled. The result is not a reconstructed bank balance. The account-summary view returns NULL for a monetary total when the account has mixed currencies; a separate account/currency view supplies valid totals.

Indexes support account/date reads and risk-event history; partial indexes cover large amounts and cross-border records. Parameterised query values prevent SQL injection through filters. Index performance still requires representative-volume EXPLAIN ANALYZE evidence. SQL constraints are the final integrity boundary even when application checks exist.

Environment configuration

.env and .env.example are different artifacts. The real .env stores machine-specific connection settings and the generated local application password; Git ignores it. The example contains placeholders and documents supported settings for another developer. It belongs in version control. Neither file is a database. The local setup creates fintrust_local for practice and fintrust_local_test for disposable automated tests under a separate application login; the administrator password is not saved.

5. AWS service choices and cost boundary

The selected services reflect workload needs. Adding every available service would increase cost and complexity without improving the capstone. No AWS profile was configured at the review, so deployed resources, account plan, credits, expiry and regional eligibility are unverified.

Service	Reason for selection	Current status and cost consideration
S3	Durable source objects, versioning, quarantine and reports	Provisioning script; requests, storage, versions and archive retrieval can be chargeable
Lambda	Short event-driven ingestion, query, scoring, alert/report handlers	Ingest/API code; later handlers incomplete. Request/duration/concurrency usage needs limits
RDS PostgreSQL	Managed relational database matching the ledger constraints	Local PostgreSQL only; private RDS provisioning and cost review pending
DynamoDB	Account/date risk lookup	Table script and local simulation; billing-mode/credit eligibility needs confirmation
API Gateway REST API	Route/auth boundary for Lambda's WSGI adapter	Planned AWS integration; API calls and related logging contribute to cost
SQS	Standard ingest DLQ; later FIFO per-account scoring queue	Ingest DLQ scripts exist; final scoring queue incomplete
IAM	Separate execution roles and scoped service actions	Scripts exist; VPC/KMS and final permission review pending
SSM Parameter Store	Runtime database configuration	Script exists; local .env replaces cloud lookup in practice mode

Service	Reason for selection	Current status and cost consideration
KMS	Customer-controlled encryption keys required by Day 1	Missing configuration; customer-managed key storage/use is not universally free
CloudWatch	Ingest/API logs, metrics and alarms	Alarm/filter scripts; dashboard, alarm actions and deployed verification pending
VPC and endpoints	Private RDS access and controlled AWS-service connectivity	No deployed VPC evidence; S3/DynamoDB gateway endpoints differ from chargeable interface endpoints
SES	High-risk email notifications	Placeholder; sandbox identities, quotas and current pricing require review
EventBridge	Scheduled monthly reports	Planned; schedule and report handler not implemented
CloudFront/private S3	Possible hosted frontend with controlled origin access	Optional future hosting; local UI avoids additional hosting services now
Step Functions	Future chunked/batch orchestration	Deferred until workflow or file-volume requirements justify it

AWS changed Free Tier arrangements for new accounts in July 2025. The applicable plan depends on account creation date and plan selection. A newer Free account plan and a paid account with remaining credits are materially different. The course's older 12-month quota table cannot establish this account's remaining allowance. [S1]

S3 and DynamoDB gateway endpoints have no additional endpoint charge; interface endpoints have different pricing. Private Lambda functions calling SSM or other AWS APIs still need an appropriate network path. Omitting a NAT Gateway does not automatically provide connectivity. Customer-managed KMS keys can incur charges. RDS storage/backups can remain chargeable while stopped, and a stopped DB instance automatically restarts after seven days. [S2, S3, S4]

Budget alerts are delayed notifications, not an instantaneous spending cap. A no-additional-charge requirement therefore blocks provisioning until the plan, credits, existing consumption, supported services and regional estimate are established. The preflight script only confirms a configured identity; it does not prove zero cost. [S5]

6. API and browser security review

Flask remains the selected framework because the detailed lesson specifies it. Framework choice does not prove latency. The frontend uses same-origin requests by default and renders user-derived text with textContent. No external JavaScript/CDN package is required for the dashboard.

Method and route	Behavior	Principal status codes
GET /	Analyst dashboard	200
GET /v1/health	PostgreSQL connectivity check	200 healthy, 503 unavailable
GET /v1/accounts	Account identifiers/types/statuses; no customer names	200, 503 dependency failure
GET /v1/overview	Counts, currency totals and daily activity	200, 503 dependency failure
GET /v1/transactions	Required UUID; from/to timestamps; limit 1-500 and offset	200, 400 invalid input, 503 database failure
GET /v1/insights	Account/date DynamoDB lookup; TTL omitted	200, 400, 404 missing score, 503 service failure

Method and route	Behavior	Principal status codes
POST /v1/upload	One multipart field named file; synchronous local ingestion only	200 completed with counts, 400 invalid file, 413 request too large, 404 local mode disabled, 503 dependency failure
OPTIONS on a route	Origin/method/header preflight	204 accepted, 403 rejected

All methods also use 403 for an unapproved Origin, 405 for an unsupported method, and 500 for an unexpected application error. Errors are JSON; responses include X-Request-ID. A 200 upload can contain quarantined rows because file processing completed with partial row acceptance. Accepted row counts include safe replays and do not claim newly inserted counts.

The future cloud upload is expected to acknowledge queued work with 202, followed by a job-status resource. That behavior is not implemented. POST /flag, /resolve and /score are later contract decisions; their absence is not masked by dummy success responses. PUT/PATCH/DELETE are not implemented in this release.

CORS allows the request's own origin and explicitly configured origins only. Approved preflight methods are GET/POST and the Content-Type header. Wildcard origins and credentialed cross-origin access are not enabled. CORS constrains browser access; it is not authentication or account authorization. The local server binds to loopback and uses synthetic data. Cloud data routes require an authenticated caller and account-level authorization before exposure; the Flask application currently has no identity model. The AWS design uses IAM for internal callers; a future human dashboard would need an appropriate identity provider and scoped authorization.

API request logs include request ID, method, path, status and elapsed milliseconds, without logging bodies or query values. Security headers include nosniff, no-store and a same-origin Content Security Policy. These controls are useful boundaries, not evidence of production readiness.

7. Reliability, logging and troubleshooting

Ingestion separates row failures from infrastructure failures. A fast batch insert is attempted; database row errors fall back to savepoints so valid records can be committed. Connection failures trigger rollback and re-raise. Completion logs include received/valid/quarantined counts, quarantine percentage and processing duration. API request logs provide request-level correlation; cross-service trace propagation is not yet complete.

S3 event notifications may be delivered more than once. The configured asynchronous ingest policy uses two retries and a standard SQS DLQ. File-level errors are not automatically repaired by retrying. Operational recovery requires diagnosis, restored dependencies, replay and reconciliation. If database inserts succeed but quarantine persistence fails, a retry relies on the ledger's duplicate protection. The two stores do not participate in a distributed transaction. [S6]

Symptom	Likely boundary	Evidence and recovery
Health 503	PostgreSQL access	Service state, host/port, application credential and network checks
Transactions 400	Request validation	UUID, timestamps, date order and pagination parameters
Insights 404	No matching daily item	Account/date key; local fixture exists only for the first sample account
Upload 400	File contract	Header, encoding, row count and filename checks
Upload 413	Request body limit	File plus multipart request size
Quarantine rows	Data quality or SQL constraint	Source row and validation/constraint reason
Windows pytest Access denied	Shared temporary-folder permissions	Local helper allocates independent scratch/cache directories

Symptom	Likely boundary	Evidence and recovery
AWS DLQ messages	Invocation exhausted retries	Original event, source-object version and CloudWatch failure log

Cloud dashboards, alarm notifications, DLQ delivery/replay, database failover and disaster-recovery evidence remain pending. Current local logs do not prove an AWS alarm has fired.

8. Testing, performance and delivery

Automated tests cover shared validation, UUID/date handling, non-finite amounts, scoring prototype behavior, multiple S3 event records, version identity, row rollback, quarantine, API status codes, pagination, CORS, request IDs, upload bounds, custom CSV upload, replay IDs, PostgreSQL constraints, currency separation and dashboard queries. Moto substitutes for AWS; PostgreSQL integration tests use the dedicated disposable database. These scopes are reported separately.

The current verified suite contains 52 passing tests with PostgreSQL enabled. The user-facing local helper selects fintrust_local_test and avoids the inaccessible shared pytest temporary directory. Browser rendering and keyboard/mobile interaction are separate verification activities; no browser automation surface was available during this review. HTTP routes and frontend assets were exercised through tests. A screenshot or visual browser acceptance is still required before final presentation.

Performance measurements must identify dataset size, warm/cold start conditions, environment, request count and percentiles. An attractive graph or use of DynamoDB is not proof of a sub-50 ms endpoint. The local smoke tool records small-sample response timings only; it is not a load benchmark. AWS cold starts, database connections and private-network routing remain unmeasured.

On 30 September 2026, seven live loopback HTTP checks returned their expected 200/400/404/405 codes. Ten serial transaction requests against the 200-transaction practice dataset measured a median of 75.08 ms and maximum of 115.42 ms. This does not measure the DynamoDB insight latency target or deployed throughput.

CI means each pushed change is checked automatically. The GitHub Actions workflow runs unit/Moto tests on pushes and pull requests; PostgreSQL integration runs on develop. JUnit reports are retained as workflow artifacts. A local passing run is not a green GitHub Actions run; remote evidence is pending.

CD can mean continuous delivery or deployment. This repository adds a manually triggered delivery workflow: unit verification, Linux x86_64 Python 3.11 packaging, environment approval and OIDC credentials, followed by an update to an existing API Lambda. It does not provision a stack or deploy the local-only uploader. Deployment environment protection and repository variables must exist in GitHub; the YAML alone does not create an approval rule. The deployment workflow is unexecuted, lacks post-deployment smoke/rollback automation, and is not a complete production release system. OIDC trust must restrict the repository and deployment environment. [S7]

Daily commits record actual verified changes using Conventional Commits. Historical dates are not fabricated. Future daily commits require a work session or a separately configured automation; a single commit cannot demonstrate weeks of incremental development.

9. Governance, limitations and production evolution

All demonstration records are synthetic. Customer names and identity numbers in a real deployment would be direct identifiers; account IDs and transaction attributes are linkable information. The public-facing dashboard excludes customer names and ID numbers. Quarantine still contains source rows and therefore requires the same classification and access control as raw files. The project is not a legal compliance certification.

Raw-file retention follows the course's 90-day transition and 2,555-day expiration assumption. Noncurrent-version age is measured differently and can extend retention beyond original creation age. RDS records, backups, quarantine and reports require approved retention and deletion workflows. DynamoDB TTL applies to operational lookup items, not the authoritative ledger. Policy/legal review is still required for real customer data.

Area	Cost-constrained foundation	Organisational implementation
Identity	Loopback demo; cloud IAM design pending	SSO/MFA, short-lived sessions, account/tenant authorization and audited role changes
Network	Private RDS design; no NAT provisioned	Reviewed VPC endpoints/routing, egress controls and segmented environments
Database	Local instance; new connection per operation	Pooled/reused connections or RDS Proxy, least-privilege SQL roles, TLS verification and tested migrations
Availability	Single local service; no failover claim	Multi-AZ RDS, capacity limits, queue backpressure and tested recovery objectives
Secrets	Ignored local .env; planned SSM	Managed rotation, restricted decrypt rights, secret inventory and access monitoring
Integrity	Source replay IDs; SQL constraints	Business transaction IDs, reconciliation and an outbox/inbox pattern across storage/queues
Detection	Rule prototype; preloaded daily insight	Approved/versioned rules, velocity windows, analyst review, outcome feedback and false-positive measurement
Observability	JSON logs and basic metric/alarm scripts	Cross-service tracing, SLOs, actionable paging and privacy-aware retention
Delivery	CI plus gated API code-update workflow	Immutable artifacts, protected branches, security scanning, staged rollout, smoke gates and rollback
Recovery	DLQ/replay design	Backup restore drills, disaster-recovery testing and approved RPO/RTO
Cost	Manual account-plan gate and local simulation	Cost allocation, budgets, anomaly detection, quotas and service ownership

Current gaps include unimplemented FIFO scoring/SES/reporting, no cloud identity enforcement, incomplete KMS/access logging/VPC configuration, no measured high-load capacity, no cross-store atomicity, no business-level deduplication, no incoming-transfer or opening-balance model, no real-time bank integration, and no proof of cloud cost eligibility. These limitations determine release scope rather than being hidden by presentation wording.

10. Verification sources and artifact map

The supplied course brief, rubric and detailed Day 1-3 lessons are the requirements basis. Conflicts and working decisions are recorded in project-reference/requirements_day1_day3.md. Formal design belongs in docs; hands-on commands belong in the learning guide. Daily progress records completed work; upcoming features holds the backlog.

S1. AWS Free Tier and account-plan distinctions:
https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/free-tier.html

S2. Gateway endpoints and private connectivity:
https://docs.aws.amazon.com/vpc/latest/privatelink/gateway-endpoints.html

S3. KMS pricing: https://aws.amazon.com/kms/pricing/

S4. RDS stop behavior and continuing charges:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_StopInstance.html

S5. Budget notification delay:
https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html

S6. S3 invocation and retry behavior: https://docs.aws.amazon.com/lambda/latest/dg/with-s3.html

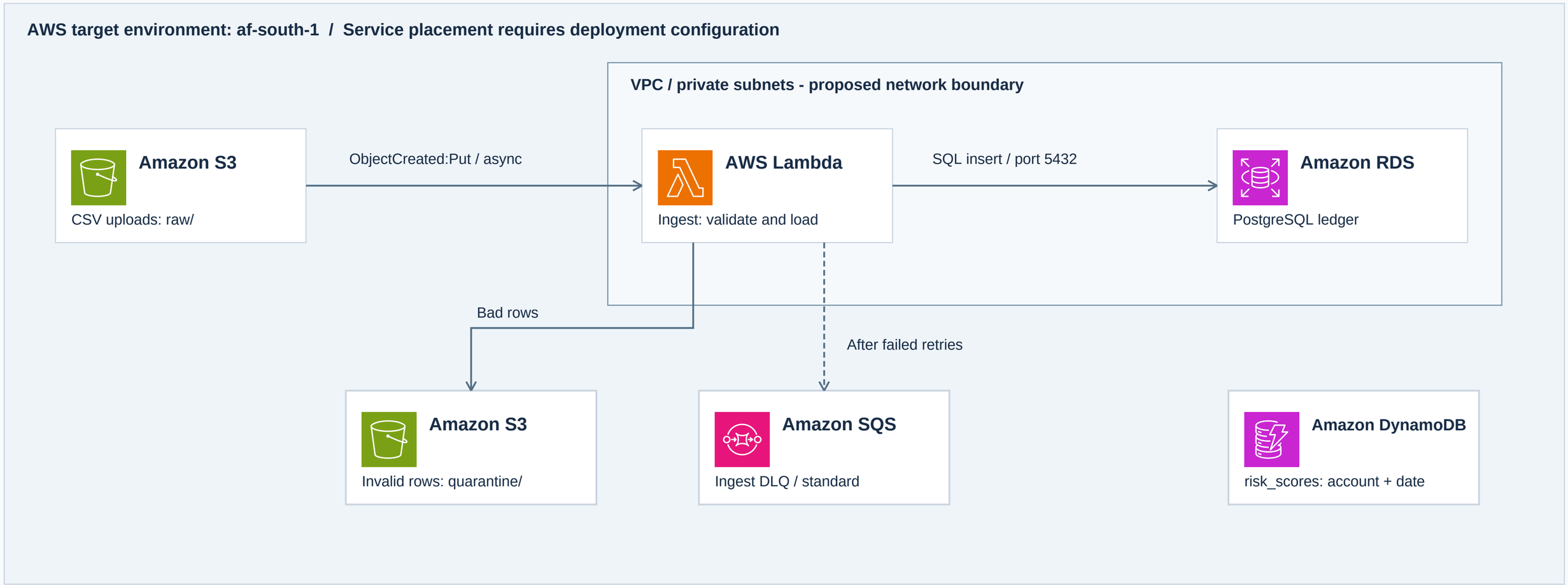
S7. GitHub AWS OIDC and environment protection:
https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-aws

S8. Flask upload limits: https://flask.palletsprojects.com/en/stable/patterns/fileuploads/

The attached diagrams include the AWS foundation and target architecture, relational ERD, activity flow, actor/use-case view and upload interaction sequence. Planned cloud components are labelled separately from the local implementation.

01 / Day 1-3 implementation

Application flow and provisioning scripts in the repository. AWS deployment has not been verified.



READ API

Flask / local + Lambda adapter

/v1/health and /v1/transactions -> PostgreSQL

/v1/insights -> DynamoDB (preloaded scores)

OPERATIONS AND SECURITY

Scripts: IAM roles, SSM parameters, CloudWatch alarms

Missing: KMS setup, access logging, VPC deployment

No automated scoring, email delivery or monthly reports yet

02 / Final capstone target

Planned architecture. Replace the direct ingest-to-RDS write when introducing FIFO; this is not a deployed-system claim.

AWS / af-south-1 | IAM per function | Customer-managed KMS keys | CloudWatch logs and alarms

Amazon S3
raw/ and quarantine/

AWS Lambda
Ingest / validation

Amazon SQS
FIFO / group by account

VPC / private subnets

AWS Lambda
Scoring + persistence

Amazon RDS
PostgreSQL

Query reads ledger; VPC attachment required

API Gateway
IAM-authorized data routes

AWS Lambda
Query / read API

Amazon DynamoDB
Daily account risk scores

AWS Lambda
Alert dispatcher

Amazon SES
Email alerts

EventBridge
Monthly schedule

AWS Lambda
Report / query ledger

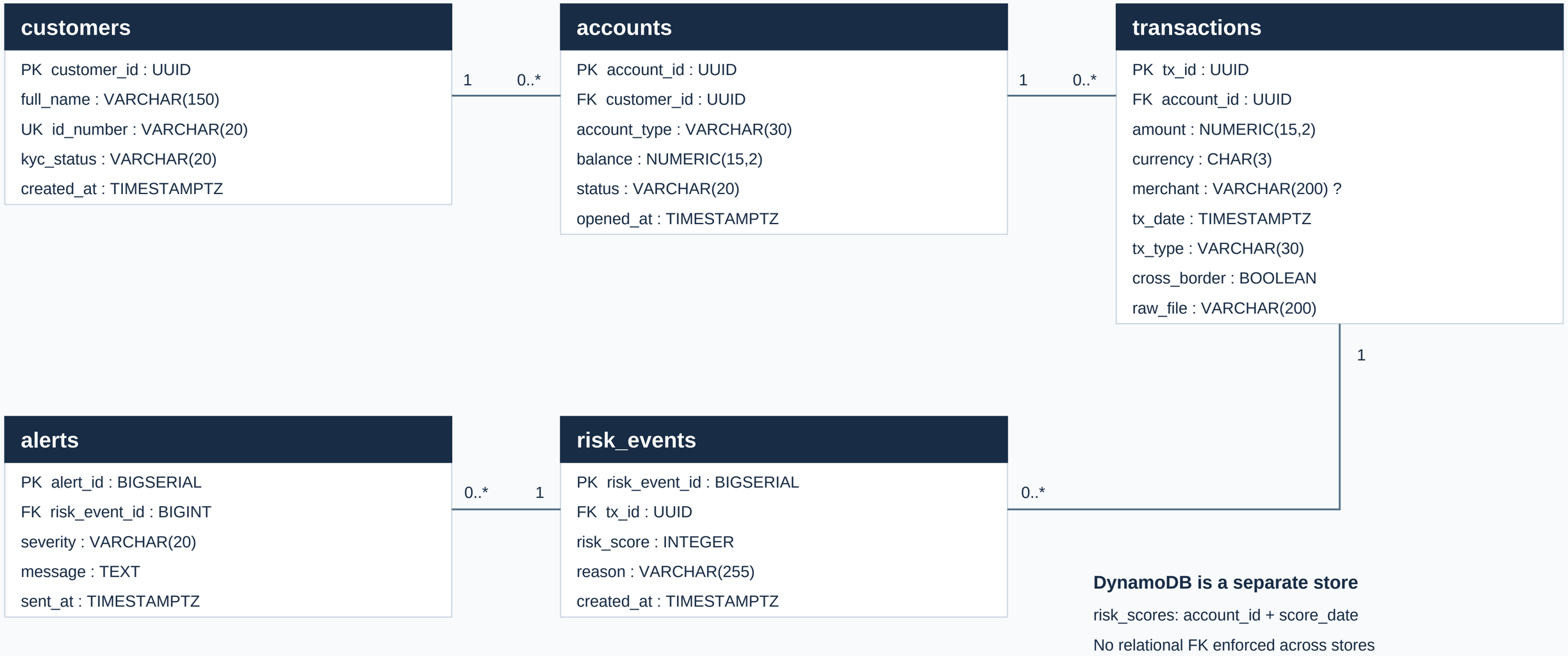
Amazon S3
CSV and JSON reports

RESILIENCE / DEPLOYMENT GAPS

- Separate ingest and scoring DLQs; replay tests
- Private AWS service access must be designed
- Budget review: no NAT; endpoints and KMS have costs
- Nightly Step Functions workflow: separate later scope

03 / PostgreSQL entity relationship diagram

Schema: sql/schema.sql | All attributes shown | PK primary key, FK foreign key, UK unique | ? nullable; others NOT NULL

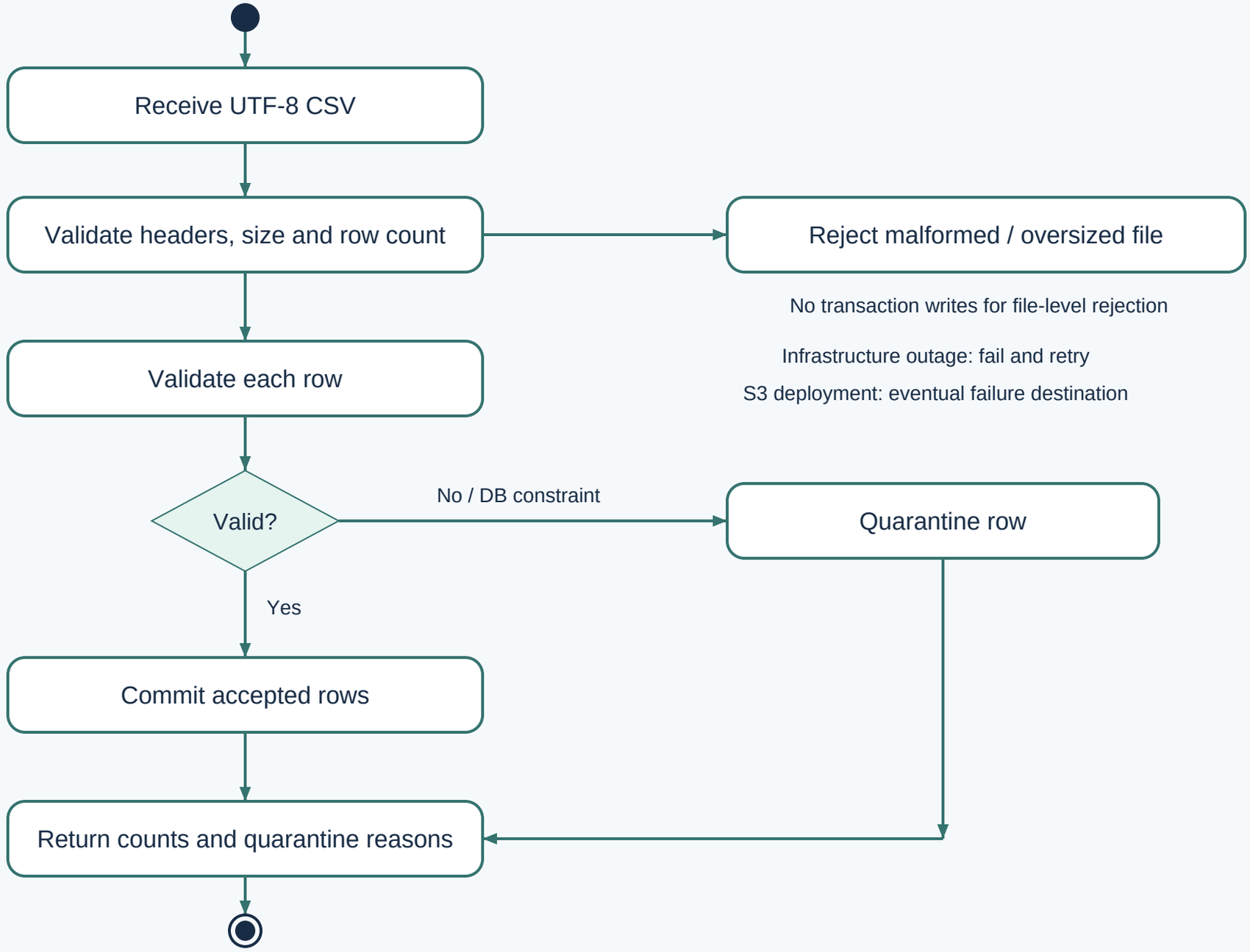


Constraints: amount > 0; risk_score 0-100; KYC, account type, currency and transaction type allowlists.

Customer/account deletes are restricted. Each child has exactly one parent; a parent may have zero or many children.

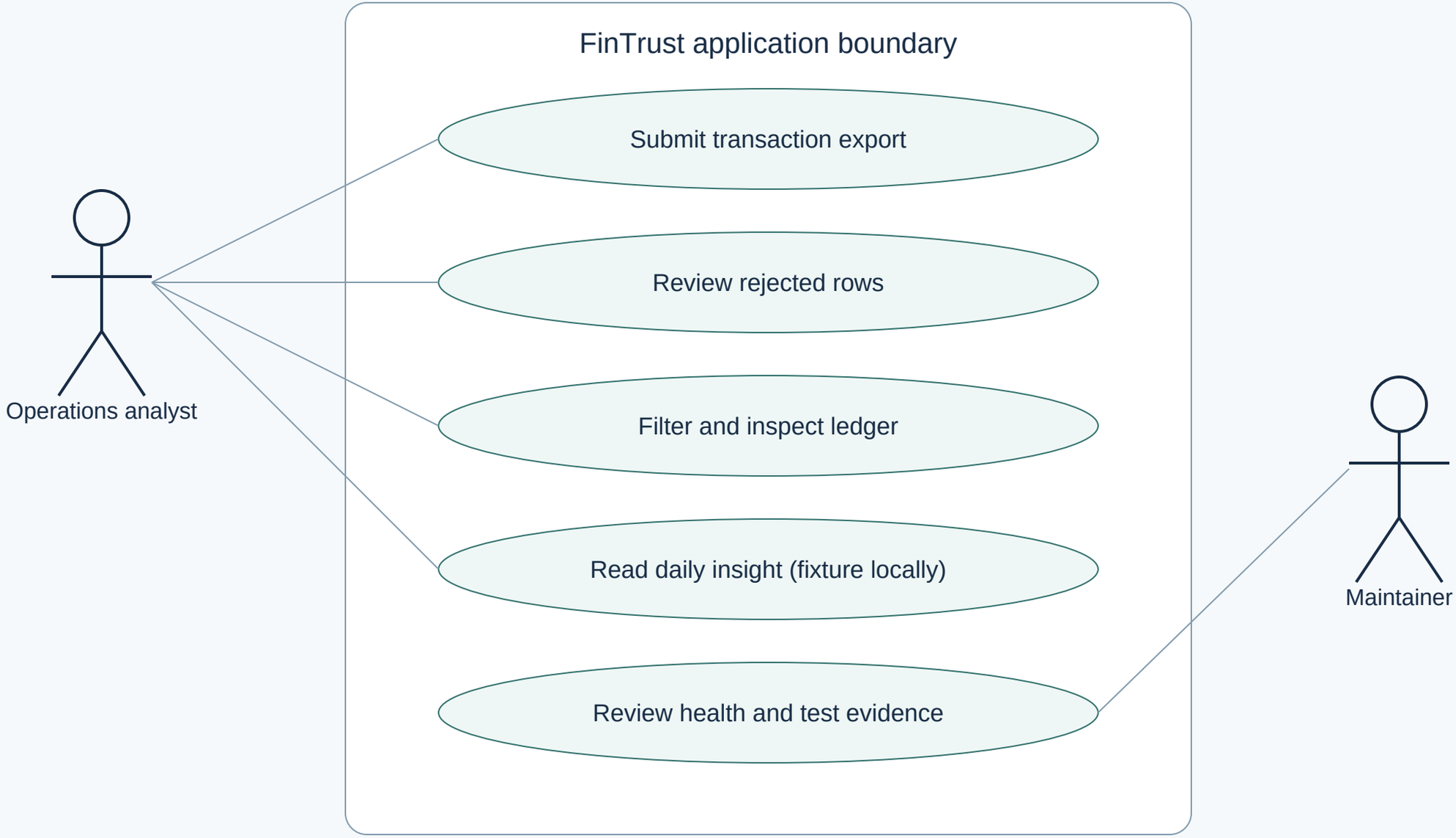
CSV processing activity

Current processing decisions; retry destinations require AWS deployment evidence.



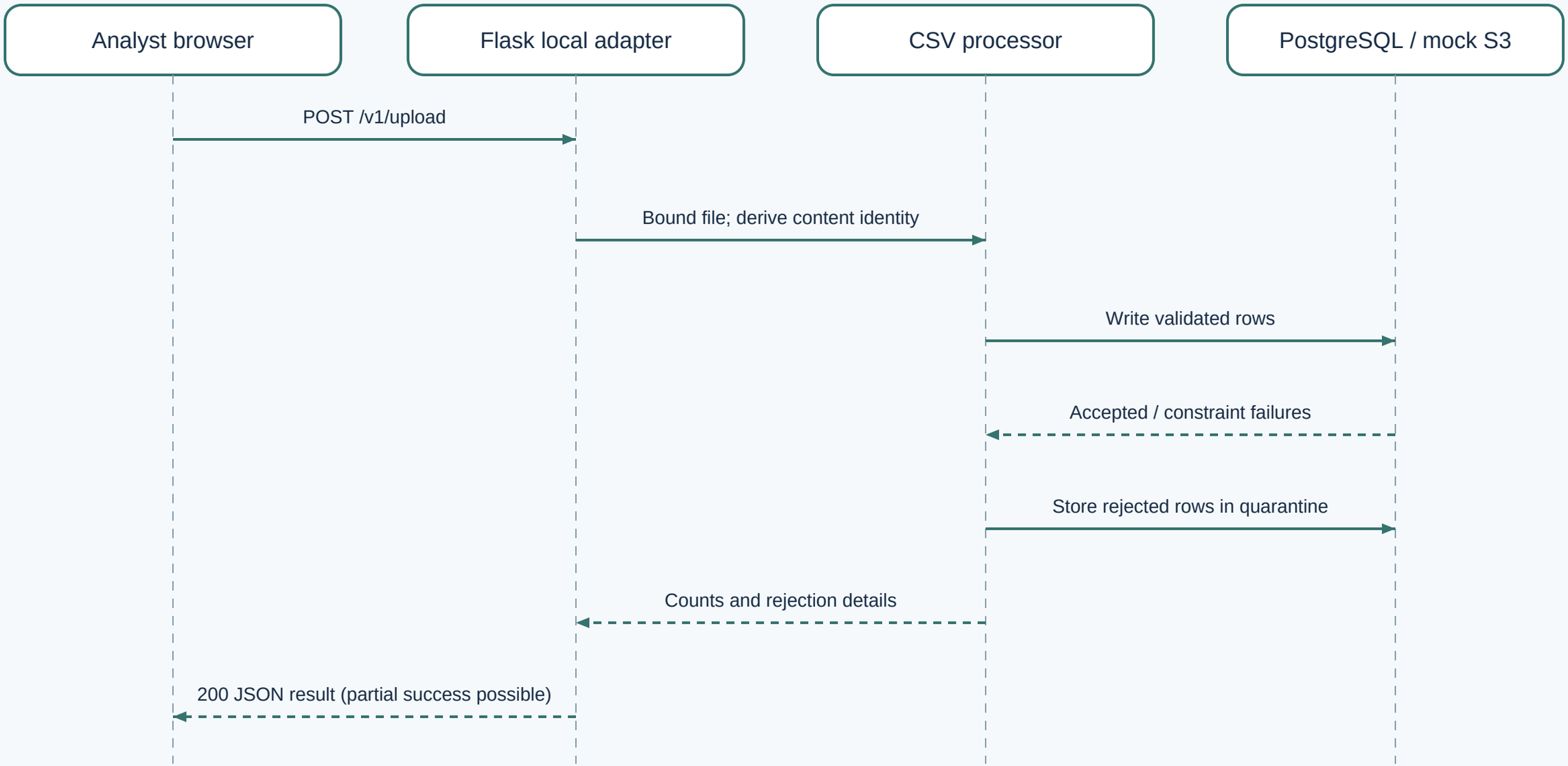
Actors and use cases

User-story view of the local foundation; production identity and access control remain planned.



Local upload interaction sequence

File-level errors return 400/413; infrastructure failures return 503. No automatic scoring follows upload.



Synchronous local upload. AWS S3-triggered ingestion has a different asynchronous entry point.